

Projektuppgift

[DT211G D2492](#)

Projektuppgift Frontendbaserad Webbutveckling

Rune Granath

OBJ



Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kuruskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 60 00. Fax: +46 (0)771 97 50 01.

DineVibe – Projektuppgift Frontendbaserad Webbutveckling Rune Granath
2026-03-16

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Rune Granath,
rugr2500@student.miun.se **Utbildningsprogram:**
Webbutveckling, 120 hp **Huvudområde:** Datateknik
Termin, år: HT, 2025

2026-03-16

Sammanfattning

Målet med detta projekt har varit att utveckla en interaktiv webbapplikation, mashupen DineVibe, som tar två stycken olika Application Programming Interfaces(API)-resurser och låter dessa interagera med varandra, med ett extra fokus på animation.

I detta fall är det två intressen, matlagning och musik, som får arbeta tillsammans för att hitta ett tips på maträtt till låten man söker på, och här har Spotify Web API för musikdata och samt Spoonacular API för receptdata använts. Utefter längden på låten får du sedan resultatet av sökningen, exempelvis snabblagade rätter för korta låtar och långkok för längre musikstycken.

Projektet har implementerats med ett fokus på asynkron programmering för anrop och en automatiserad utvecklingsmiljö, vilket huvudsakligen har stöttats av byggverktyget Vite.

Slutsatsen är att det finns oändliga möjligheter för att kunna skapa nischade webbtjänster.

Innehållsförteckning

Sammanfattning iii Förord iv Terminologi vi

1	Introduktion	/	Inledning
.....	1	1.1	Bakgrund och
problemmotivering.....	1	1.2	
Övergripande syfte / Högnivåproblemformulering.....	2		
1.3			Avgränsningar
.....	2	1.4	
Konkreta och verifierbara mål / Detaljerad problemformulering	2	1.5	
Översikt.....			
.....3	1.6		Författarens
bidrag.....		3	
2 Teori / Bakgrundsmaterial.....			
4	2.1	Definition	av termer och förkortningar
.....	4	2.1.1	Exempel på rubriknivå 3
.....	5	2.2	Att referera eller
citera	5	2.3	

Källförteckning	och	källhänvisningar
.....5	2.4	Automatiskt numrerade
källhänvisningar.....6		2.5
Illustrationer.....7	2.6	Matematiska formler
.....8		
3		
Metod.....		
... 9 4	Konstruktion /	Lösningssalternativ
.....10		5
Resultat.....		
11 6	Slutsatser /	Analys / Diskussion
.....12	Källförteckning	13
Bilaga A: Dokumentation av egenutvecklad programkod	14	Exempel på
underrubrik		14

Terminologi Akronymer och förkortningar

- **API** *Application Programming Interface*. Ett gränssnitt som gör det möjligt för olika mjukvaruapplikationer att kommunicera och utbyta data med varandra.
- **DOM** *Document Object Model*. En programmeringsgränssnitt för HTML-dokument som representerar sidans struktur som ett träd av objekt, vilket möjliggör manipulation via JavaScript.

- **ES6+ *ECMAScript 2015*** (och senare versioner). Den moderna standarden för JavaScript-språket som introducerade funktioner som pilar, klasser och moduler.
- **JSON *JavaScript Object Notation***. Ett lättviktigt format för datalagring och kommunikation som är enkelt för både människor att läsa och maskiner att tolka.
- **SCSS/Sass *Syntactically Awesome Style Sheets***. Ett förbehandlingsspråk (preprocessor) för CSS som tillåter variabler, nästling och mer avancerad logik i designarbetet.

Tekniska begrepp och verktyg

- **Asynkron programmering** En programmeringsmetod (ofta via `async/await` i JavaScript) som tillåter att tidskrävande processer, såsom API-anrop, körs i bakgrunden utan att frysa programmets övriga exekvering.
- **GitHub Actions** Ett verktyg för automatisering av arbetsflöden, i detta projekt använd för *Continuous Deployment* (CD) för att automatiskt bygga och publicera webbplatsen.
- **JSDoc** Ett märkspråk som används för att kommentera JavaScript kod, vilket möjliggör automatisk generering av teknisk dokumentation.
- **Mashup** En webbapplikation som kombinerar data eller funktionalitet från två eller flera externa källor för att skapa en ny tjänst.
- **Vite** Ett modernt byggverktyg och utvecklingsserver som optimerar laddningstider och paketering av källkod under utvecklingsprocessen.

v

Projektuppgift Frontendbaserad Webbutveckling – DineVibe Rune Granath
2026-03-18

1 Introduktion

Detta inledande kapitel ger en övergripande bild av projektet DineVibe. Här presenteras bakgrunden till varför applikationen skapats, vilka problem den ämnar lösa samt vilka mål och avgränsningar som funnits under utvecklingsprocessen.

1.1 Bakgrund och problemmotivering

Idén till DineVibe föddes ur behovet av att förenkla vardagsbeslut genom att kombinera nytta med nöje. Många upplever beslutsångest och monotoni kring vad de ska laga till middag, samtidigt som musik är en central del av deras vardag och ofta kan visa hur man känner för stunden. Genom att föra dessa samman skulle det kunna fylla den processen med mer lust och mat. Val av mat till mående för ett mer kreativt beslut. Personer som kan tänkas använda appen är mest troligt yngre användare med god vana för nya sorters appar, vilket då kräver ett modernt utseende och nymodig design.

Problemet som ska lösas är alltså att automatisera beslutsfattandet, medan tekniskt sett utmaningen ligger i att få två helt olika API-tjänster, Spotify och Spoonacular, att utbyta information på ett logiskt sätt.

1.2 Övergripande syfte / Högnivåproblemformulering Projektets syfte är att utveckla en fungerande webbapplikation som demonstrerar färdigheter i att hämta, transformera och presentera data från externa JSON-källor. Vidare syftar projektet till att använda en professionell och automatiserad arbetsprocess med moderna verktyg som Vite och GitHub Actions.

1.3 Avgränsningar

För att hålla projektet inom tidsramen har följande avgränsningar gjorts:

- Inga ramverk (som React eller Angular) används.
- Systemet hanterar inte användarinloggning eller har en större nivå av säkerhetstänk kring API-nycklar.
- Sökningar är begränsade till låttitlar och bara ett svar från receptkategorier, och endast låtarnas längd hämtas i APIet.

1.4 Konkreta och verifierbara mål / Detaljerad problemformulering

Målet med uppgiften har främst varit att två eller fler API-tjänster ska kunna kommunicera med varandra, i en automatiserad utvecklingsmiljö där resultatet ska vara en app-liknande sida som även ska innehålla olika animationer.

2 Teori / Bakgrundsmaterial

Detta kapitel definierar de tekniska begrepp och verktyg som ligger till grund för projektets genomförande. En förståelse för dessa tekniker är nödvändig för att kunna tolka applikationens arkitektur och funktionalitet.

2.1 JSON-baserade webbtjänster och API

Ett *Application Programming Interface* (API) fungerar som ett standardiserat gränssnitt som möjliggör kommunikation mellan två oberoende mjukvarusystem. I detta projekt används REST-baserade API:er som levererar data i formatet *JavaScript Object Notation* (JSON). JSON är ett textbaserat format för datalagring som bygger på nyckel-värde-par, vilket gör det enkelt för JavaScript att parsas (tolka) till objekt.

Genom att använda JSON kan DineVibe hämta strukturerad information från externa servrar, såsom låttitlar från Spotify [1] eller ingredienslistor från Spoonacular [2], och integrera dessa i det lokala användargränssnittet.

2.2 Asynkron programmering och Fetch API För att skapa en modern och följsam webbapplikation används asynkron programmering. Traditionell synkron programmering exekverar kod rad för rad, vilket innebär att webbläsaren "fryser" medan den väntar på svar från en extern server. Genom att använda *fetch-metoden* och nyckelorden *async* och *await* kan anropen ske asynkront [3]. Detta innebär att applikationen fortsätter att köras medan data hämtas i bakgrunden, vilket resulterar i en oavbruten användarupplevelse.

2.3 Mashup-arkitektur

Begreppet "mashup" avser en webbapplikation som kombinerar innehåll från mer än en källa för att skapa en ny, integrerad tjänst. DineVibes arkitektur bygger på att transformera data mellan två källor: metadata från Spotify, i form av låtlängd i millisekunder, hämtas först och räknas om till minuter. Låtlängden normaliseras sedan till en längdfaktor mellan 0 och 1, där kortare låtar får ett lägre värde och låtar på fem minuter eller längre får maxvärdet 1. Den normaliserade låtlängden används därefter för att välja en receptrelaterad sökterm till Spoonacular, exempelvis "snack" för kortare låtar och "slow cook" för längre låtar. Detta samarbete mellan API:erna är kärnan i projektets logik.

2.4 CSS-animeringar och användarupplevelse (UX) Animeringar i ett gränssnitt handlar inte enbart om estetik, utan fungerar som visuell feedback till användaren. Genom att använda CSS-transitions och keyframes kan applikationen kommunicera statusförändringar, såsom att data laddas eller att ett resultat har hittats. I projektet används animeringar även för att förstärka den tematiska kopplingen mellan musikens tempo och receptets karaktär.

2.5 Automatiserade arbetsprocesser med Vite och Node.js samt publicering

Node.js fungerar som körmiljö för projektets byggverktyg, Vite [4]. Vite

- **Module Bundling:** Kombinerar flera JavaScript-filer till en optimerad fil för att minska antalet HTTP-anrop.
- **Transpiling:** Säkerställer att modern JavaScript-kod fungerar även i äldre webbläsare.
- **Minifying:** Komprimerar källkod genom att ta bort blanksteg och korta ner variabelnamn, vilket sänker sidans laddningstid.

Själva publiceringen konfigurerades sedan och kopplades till GitHub pages och med hjälp av GitHub actions vilket gjorde att 'push' av kod automatiskt triggade bygget och en uppdatering av sidan.

2.6 CSS-preprocessorer (Sass/SCSS)

Sass (*Syntactically Awesome Style Sheets*) är en preprocessor som utökar funktionaliteten i standard-CSS. Genom att använda filändelsen *.scss* kan utvecklaren använda variabler för färger och typsnitt, nästla selektorer för bättre överblick och dela upp koden i moduler. Detta skapar en mer lättunderhållen och skalbar kodbas.

2.7 Versionshantering och CI/CD

Git används för versionshantering, vilket möjliggör spårning av ändringar i källkoden och samarbete. Projektet tillämpar även *Continuous Integration/Continuous Deployment* (CI/CD) via GitHub Actions. Detta innebär att varje gång ny kod skickas till GitHub, startas en automatiserad process som bygger, optimerar och publicerar webbplatsen på GitHub Pages.

2.8 Designprinciper för mobila webbapplikationer

För att ge DineVibe en autentisk "app-känsla" har designprocessen utgått från etablerade mönster inom mobil gränssnittsdesign (UI). Inspiration har hämtats via moodboards på tjänster som Pinterest för att identifiera en modern och funktionell estetik, medan matchande färgscheman har letats på Colorhunt.co. Följande principer har tillämpats:

- **Kortbaserad layout (Card-Based):** Istället för traditionella listor presenteras recept och låtar som separata "kort". Detta skapar en ren och modulär struktur som är enkel att skanna visuellt och fungerar väl i responsiva flöden.
- **Glassmorphism:** En modern designstil som använder halvgenomskinliga bakgrunder med suddighetseffekter (*backdrop-*

operativsystem som iOS och Windows 11.

- **Dashboard-vy:** Applikationens huvudvy är utformad som en instrumentbräda där data från Spotify (spelaren) och Spoonacular (receptet) presenteras sida vid sida för att tydliggöra kopplingen i denna mashup.

2.9 Begränsningar i externa API-resurser

En kritisk aspekt vid utveckling av mashups är beroendet av tredjepartstjänsters tillgänglighet och dokumentation. Under projektets gång identifierades en teknisk begränsning i Spotifys ekosystem.

Tidigare erbjöd Spotify en endpoint kallad *Audio Features* (för analys av t.ex. "energy" och "danceability"), men i december 2024 begränsades åtkomsten till denna funktion drastiskt. Spotify kräver numera en manuell granskning och godkännande av applikationer för att få tillgång till dessa data. För att säkerställa att projektet förblev funktionellt och skalbart utan dessa begränsningar, valdes en alternativ logik. Istället för energinivåer baseras matchningen på en kombination av låtens tidslängd (duration), vilket är publikt tillgängligt via Spotifys sök-API. Detta för att prioritera applikationens driftsäkerhet och tillgänglighet för slutanvändaren.

3 Metod

Arbetet med DineVibe inleddes med en planeringsfas där fokus låg på att hitta API-resurser som kunde skapa en meningsfull och intressant interaktion. Genom förstudien valdes Spotify Web API och Spoonacular Food API, då båda är någorlunda erkända tjänster som erbjuder välstrukturerade data i JSON-format och en dokumentation som tydligt beskriver hur asynkrona anrop ska utföras.

Efter att de tekniska förutsättningarna fastställts påbörjades designprocessen med ett sökande efter inspiration och trender för design. Med hjälp av moodboards på Pinterest efterforskades moderna designtrender, vilket ledde fram till beslutet att implementera en kortbaserad layout och "Glassmorphism" för att ge webbapplikationen en autentisk och lyxig känsla.

Själva systemutvecklingen genomfördes med en iterativ programmeringsmetod i JavaScript, där fokus låg på att bygga robusta asynkrona funktioner med Fetch API och att arbeta främst med Javascriptfilen för att skapa huvudinnehållet på sidan. AJAX-anrop (Asynchronous JavaScript and XML) användes som den primära metoden för datakommunikation för att säkerställa att användargränssnittet blev interaktivt medan data hämtas. Denna metodik möjliggjorde att data kunde hämtas och transformeras i bakgrunden, vilket är avgörande för att skapa den efterfrågade "webb-app"-känslan utan störande sidomladdningar.

En central del av utvecklingsmetoden var skapandet av den logiska bryggan mellan de två API-tjänsterna, där låtens längd i millisekunder konverterades till minuter för att styra receptsökningen och visa användaren en procentuell längdfaktor utifrån låtens längd.

Genom att använda byggverktyget Vite automatiserades optimeringen av källkoden, såsom minifiering och modulhantering, vilket säkerställde att den publika versionen av webbplatsen alltid var prestandaoptimerad.

För att hantera kodens versionshantering användes Git i kombination med en automatiserad kedja för att bygga upp sidan automatiskt till GitHub pages via GitHub Actions. Github startar då upp en virtuell dator vid bygget som installerar samma Node.js-miljö som din och fullföljer det.

Slutligen kvalitetssäkrades projektet genom att använda JSDoc för teknisk dokumentation av samtliga funktioner, samt genom tester där olika sökscenarier kördes för att verifiera att algoritmerna för tidsmatchning och de villkorsstyrda animationerna fungerade korrekt i enlighet med projektets uppsatta mål.

Lösningalternativ

Den tekniska konstruktionen av DineVibe vilar på en arkitektur där JavaScript fungerar som motorn som driver interaktionen mellan användar gränssnittet och de externa API-resurserna. Vid applikationens start initieras ett sökformulär som fångar upp användarens input och skickar denna vidare till en dedikerad funktion för kommunikation med Spotify Web API. Genom att använda moderna JavaScript-konstruktioner som *async* och *await* genomförs AJAX-anrop mot Spotifys sök-endpoint, där ett JSON-svar inne hållande metadata för låtar returneras.

```
const music = await getMusicMood(searchTerm);
```

Figur 4.1. Konstanten music väntar på funktionen getMusicMood

Konstruktionens hjärta är den transformeringslogik som tar fältet *duration_ms* från Spotifys data och räknar om detta värde från millisekunder till minuter genom division med 60 000. Låtens längd normaliseras sedan mot ett valt maxvärde på fem minuter. Resultatet sparas som **lengthScore**, ett värde mellan 0 och 1, där kortare låtar får ett lägre värde och låtar på fem minuter eller längre får maxvärdet 1. I gränssnittet visas detta som en procentuell längdfaktor genom **lengthScore * 100**.

```
const minutes = track.duration_ms / 1000 / 60;  
  
let lengthScore = minutes / 5;  
if (lengthScore > 1) lengthScore = 1;
```

Figur 4.2. Beräkning av lengthScore genom konvertering och normalisering av tidsdata.

Värdet **lengthScore * 100** presenteras sedan för användaren som en procentuell längdfaktor i gränssnittet. . 6

Projektuppgift Frontendbaserad Webbutveckling – DineVibe Rune Granath
2026-03-18

Genom denna logiska brygga transformeras musikens egenskaper till en sökterm för recept. Variabeln *foodQuery* fungerar som länken mellan de två oberoende tjänsterna; den bär med sig instruktioner baserade på Spotify-datan in i anropet mot Spoonacular API.

Ett värde ($\text{music.minutes} * 100$) presenteras sedan för användaren som en procentuell 'längdfaktor' i gränssnittet.

För att optimera prestandan och minska antalet onödiga anrop har konstruktionen försetts med felhantering som validerar att en låt faktiskt hittats innan sökningen efter recept påbörjas.

Det visuella gränssnittet är uppbyggt med en kortbaserad layout där varje recept presenteras i ett dynamiskt genererat HTML-element. För att skapa en levande

och responsiv upplevelse har CSS-animeringar integrerats direkt i logiken; genom att använda villkorsstyrda klasser i JavaScript triggas olika visuella effekter beroende på resultatet av sökningen. Exempelvis har en specifik CSS-animation med "dansande sniglar" konstruerats för att visas vid recept med lång tillagningstid:

```
if (isLongest) {
  const snailsContainer = document.createElement("div");
  snailsContainer.classList.add("snails-container");

  for (let i = 0; i < 6; i++) {
    const snailEl = document.createElement("div");
    snailEl.classList.add("dancing-snail");
    snailEl.textContent = "🐌";
    snailsContainer.appendChild(snailEl);
  }

  document.body.appendChild(snailsContainer);
}
```

Figur 4.4. I JavaScript: de längsta låtarna triggas en animation där sniglar läggs till i en for-loop.

Med hjälp av *@keyframes* och *transform: scaleX(1)* och *scaleX(-1)* i CSS "vänder" de sig i en tidslinje.

Rent stilistiskt är applikationen konstruerad med SCSS, vilket har möjliggjort användandet av avancerade tekniker som *backdrop filter: blur* för att uppnå den tänkta Glassmorphism-effekten.

```
@mixin glass-effect($blur: 10px, $bg: rgba(255, 255, 255, 0.05)) {
  background-color: $bg;
  -webkit-backdrop-filter: blur($blur);
  backdrop-filter: blur($blur);
}
```

Figur 4.5. Backdrop-filter fungerar bara på det som finns bakom element. 10px är graden av suddighet och 0.05-värdet är opaciteten.

API-nycklar och konfigurationsvärden har brutits ut till en separat konfigurationsfil för att skilja konfiguration från applikationslogik. Detta förbättrar kodens struktur och underhållbarhet, men innebär inte full säkerhet eftersom nycklar som används i frontend fortfarande kan läsas i webbläsaren. I

en produktionslösning bör känsliga nycklar hanteras via backend eller serverless-funktion.

När det var dags att publicera till Github pages med Github actions skapades en .yml-fil med olika instruktioner, däribland npm run build den viktigaste, för att kunna automatisera bygget och publiceringen med en enda kod-'push' till Github.

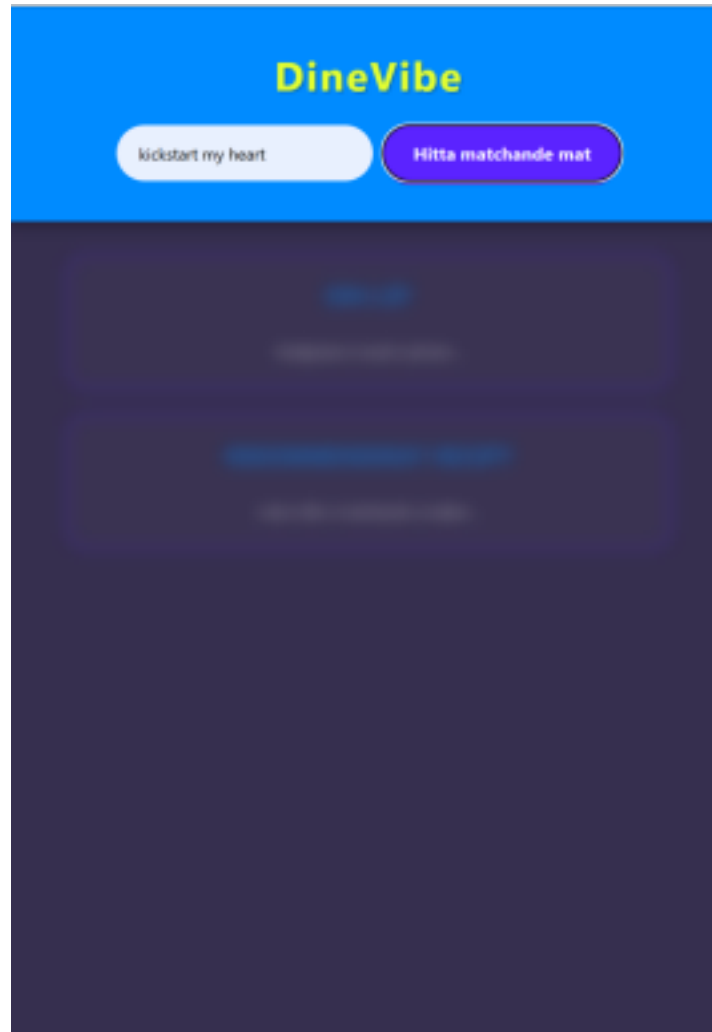
Slutligen har hela konstruktionen paketerats och optimerats med hjälp av byggverktyget Vite, som vid varje publicering minifierar JavaScript-filerna och kompilerar SCSS-koden till standard-CSS för maximal kompatibilitet och snabba laddningstider i webbläsaren.

5 Resultat

I detta kapitel presenteras den färdiga webbappen och hur de tekniska lösningarna som beskrivits i kapitel 4 fungerar i praktiken för slutanvändaren.

För att säkerställa att integrationen mellan Spotify API och Spoonacular API fungerar korrekt har systemet testats med olika typer av musik. Resultaten visar att låtarnas längd omvandlas till en normaliserad längdfaktor, vilken styr valet av receptkategori.

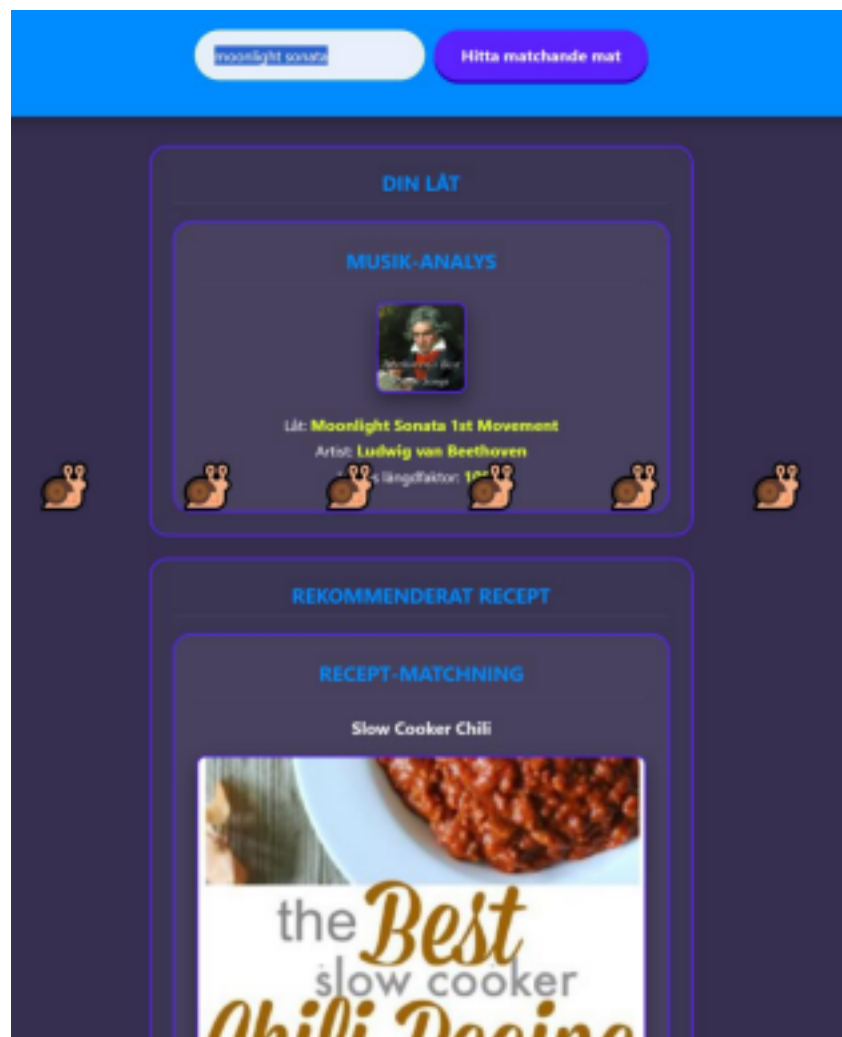
Appen har ett enhetligt gränssnitt som följer den planerade designstrategin med



“Glassmorphism”:

Figur 5.1. Den tvingade laddningssekvensens utseende efter en sökning har gjorts.

En sökning på en lång låt visar ett långkoks-recept och triggas de dansande sniglarna över skärmen:



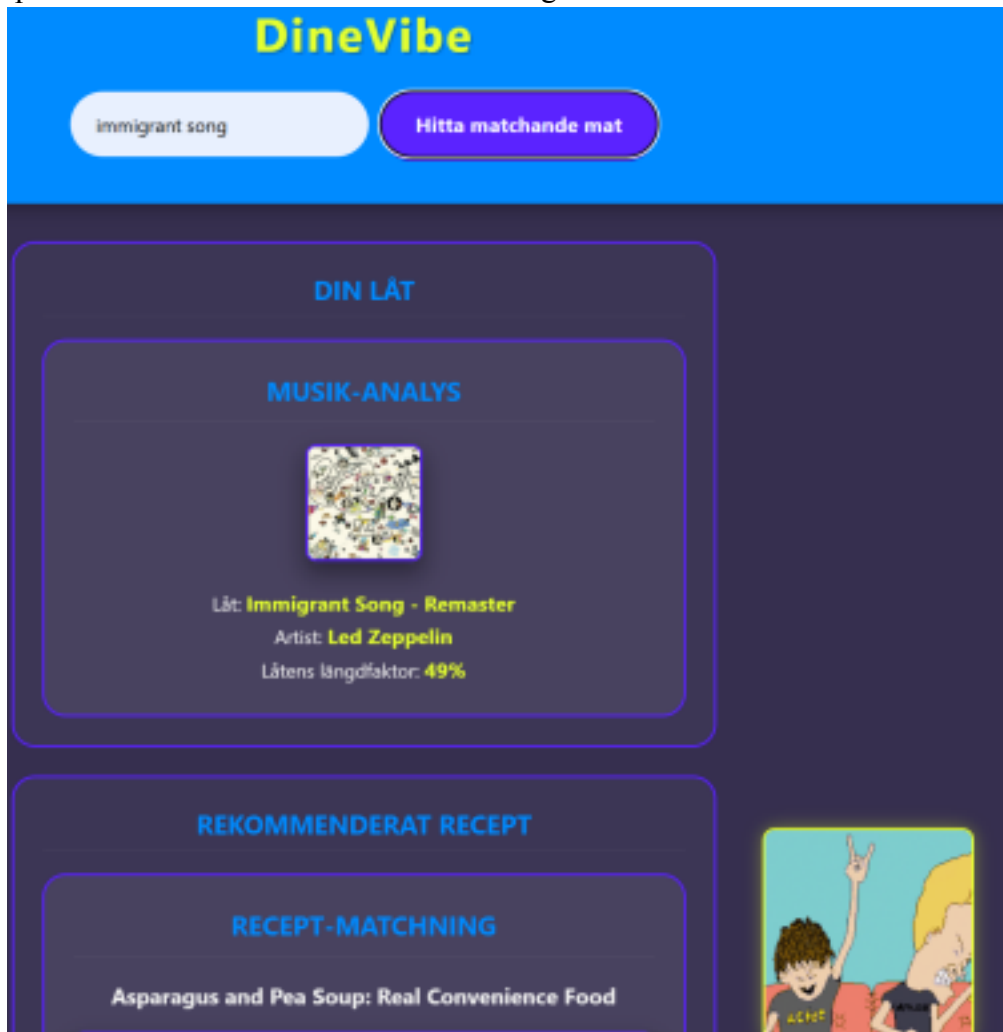
2026-03-18

Figur 5.2. Låtar där lengthScore är max

Resultaten från sökningarna ger slutligen användaren en bild från låtsökningen med namn och artist med en normaliserad 'längdfaktor' i %, och ett matchat recept med bild och länk.

2026-03-18

Och de kortaste låtarna ger oss ett snabbt recept och triggas visningen av en specifik animation i gränssnittets nedre hörn.



Figur 5.3. visar hur kortare låtar leder till recept inom snabbare kategorier och samtidigt aktiverar en separat animation. Detta demonstrerar både API-kopplingen mellan låtlängd och receptsökning samt kravet på villkorsstyrda animationer i gränssnittet.

Den färdiga webbapplikationen uppfyller projektets huvudsakliga tekniska krav genom att kombinera två JSON-baserade webbtjänster i en gemensam mashup. Spotify Web API används för att hämta musikdata utifrån användarens sökning, medan Spoonacular API används för att hämta ett recept som matchas mot den information som hämtats från Spotify. Kopplingen mellan API sker genom att låtens längd omvandlas till en normaliserad längdfaktor, som därefter används för att välja en receptrelaterad sökterm. På så sätt påverkar resultatet från den första webbtjänsten vilket innehåll som hämtas från den andra.

Applikationen har byggts utan frontend-ramverk och använder JavaScript för att skapa och uppdatera större delen av gränssnittets innehåll dynamiskt. Användarens sökning hanteras med asynkrona fetch-anrop, vilket gör att data kan hämtas och presenteras utan sidomladdning. JavaScript-koden har dokumenterats med JSDoc, vilket gör funktionernas syfte, parametrar och returvärden tydligare och bidrar till att koden blir lättare att underhålla.

Projektet använder även en automatiserad arbetsprocess baserad på Node.js och Vite. Vid publicering byggs och optimeras webbplatsen automatiskt, och GitHub

Actions används för att publicera den färdiga versionen till GitHub Pages. Styling har skrivits med SCSS, där bland annat variabler, nästling och återanvändbara strukturer används för att skapa en mer organiserad kodbas. Webbplatsens design är responsiv och bygger på en kortbaserad layout med glassmorphism-inspirerade visuella element, vilket ger applikationen ett tydligt app-liknande uttryck.

Utöver den grundläggande API-funktionaliteten innehåller gränssnittet flera animationer som är kopplade till användarens sökresultat. En laddningsanimation visas medan data hämtas, och olika visuella effekter aktiveras beroende på låtens längdfaktor. Animationerna fungerar därför både som återkoppling till användaren och som ett sätt att förstärka kopplingen mellan musikens längd och receptets karaktär. Sammantaget visar resultatet att projektets centrala mål har uppnåtts: att skapa en publicerad, responsiv och interaktiv webbapplikation där flera webbtjänster samverkar genom JavaScript och en automatiserad arbetsprocess.

6 Diskussion

Jag tycker appen som helhet med sin design och funktion blev lyckad sett till att detta är en första kurs i frontendbaserad webbutveckling. Även om slutprodukten är relativt enkel i sin funktionalitet står den på en stabil teknisk grund.

Efter upptäckten att Spotify begränsat den data man får från deras API blev beslutet att simplificera resultatet användaren fick ut från sin sökning något trist, men det var också nödvändigt för att man över huvud taget skulle kunna använda tjänsterna tillsammans. Om utvecklandet hade fortsatt i två veckor till hade fokus legat främst på implementering av mer data från Spoonaculars API,

till exempel vinmatchning eller ingredienslistor där sökresultaten hade varit mer obegränsade och randomiserade för att få variation och liv i användandet av tjänsten.

Annars har arbetet, främst med automatisering, visat prov på hur man faktiskt kommer börja jobba “på riktigt” som en utvecklare, och allt krångel med diverse inställningar för att få det på plats har gett en bra grund och förståelse för att sedan kunna gå vidare till mer hela lösningar, även om dem säkerligen kommer med sina egna sorters krångel.

7 Källförteckning

[1] **Spotify for Developers.** Web API Reference.

<https://developer.spotify.com/documentation/web-api> (Hämtad 2026-03-25).

[2] **Spoonacular API.** Documentation. <https://spoonacular.com/food-api/docs> (Hämtad 2026-03-25).

[3] **MDN Web Docs.** Using the Fetch API.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (Hämtad 2026-03-25).

[4] **Vite.** Next Generation Frontend Tooling. <https://vitejs.dev/guide/>
(Hämtad 2026-03-25).